Lab 4: Regression and Classification Evaluation Metrics

Part 1: Comprehensive Study of K-Nearest Neighbours (KNN) Classification using Breast Cancer Dataset

Student ID: 2547119
Dataset: Breast Cancer Wisconsin (Diagnostic) — `brca.csv`
Objective: Implement KNN classification, tune K using heuristic + cross-validation, evaluate using classification metrics, and compare with regression evaluation metrics from Lab 3.

Imports and Setup

We import all libraries upfront so dependencies are explicit and the notebook can be re-run cleanly.

- **numpy / pandas** — data manipulation
- **matplotlib / seaborn** — visualisation
- **sklearn** — preprocessing, model, metrics, validation

In [2]:
```
# ── Standard Library ──────────────────────────────────────────────
import warnings
warnings.filterwarnings('ignore')

# ── Data Handling ─────────────────────────────────────────────────
import numpy as np
import pandas as pd

# ── Visualisation ─────────────────────────────────────────────────
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from matplotlib.colors import ListedColormap

# ── Sklearn Preprocessing ─────────────────────────────────────────
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score, Strat
from sklearn.decomposition import PCA

# ── Sklearn Model ─────────────────────────────────────────────────
from sklearn.neighbors import KNeighborsClassifier
```

```
# ── Sklearn Metrics ──────────────────────────────────────────────────
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, roc_auc_score, classification_report
)

# ── Global Plot Style ────────────────────────────────────────────────
sns.set_theme(style='whitegrid', palette='muted', font_scale=1.1)
plt.rcParams['figure.dpi'] = 110
plt.rcParams['axes.spines.top']   = False
plt.rcParams['axes.spines.right'] = False

print("All libraries imported successfully.")
print(f"  numpy      : {np.__version__}")
print(f"  pandas     : {pd.__version__}")
import sklearn; print(f"  sklearn    : {sklearn.__version__}")
import matplotlib; print(f"  matplotlib : {matplotlib.__version__}")
import seaborn; print(f"  seaborn    : {seaborn.__version__}")
```

```
All libraries imported successfully.
  numpy      : 2.5.1
  pandas     : 3.0.3
  sklearn    : 1.9.0
  matplotlib : 3.11.0
  seaborn    : 0.13.2
```

Task 1 — Data Preparation

> **Goal:** Load the raw CSV, understand its structure, check data quality, encode the target, apply feature scaling, and justify every step.

1.1 Load & Initial Exploration

In [3]:

```
# ── Load CSV ─────────────────────────────────────────────────────────
# index_col=0  →  the first column is a row index from the original R export
df = pd.read_csv('brca.csv', index_col=0)

print("=" * 55)
print("DATASET OVERVIEW")
print("=" * 55)
print(f"Shape           : {df.shape[0]} rows  ×  {df.shape[1]} columns")
print(f"Feature columns : {df.shape[1] - 1}  (30 numeric)")
print(f"Target column   : 'y'  →  B (Benign) | M (Malignant)")

print("\nColumn names:")
print(df.columns.tolist())
```

```
==========================================================
DATASET OVERVIEW
==========================================================
Shape           : 569 rows  ×  31 columns
Feature columns : 30  (30 numeric)
Target column   : 'y'  →  B (Benign) | M (Malignant)

Column names:
['x.radius_mean', 'x.texture_mean', 'x.perimeter_mean', 'x.area_mean', 'x.smo
othness_mean', 'x.compactness_mean', 'x.concavity_mean', 'x.concave_pts_mean'
, 'x.symmetry_mean', 'x.fractal_dim_mean', 'x.radius_se', 'x.texture_se', 'x.
perimeter_se', 'x.area_se', 'x.smoothness_se', 'x.compactness_se', 'x.concavi
ty_se', 'x.concave_pts_se', 'x.symmetry_se', 'x.fractal_dim_se', 'x.radius_wo
rst', 'x.texture_worst', 'x.perimeter_worst', 'x.area_worst', 'x.smoothness_w
orst', 'x.compactness_worst', 'x.concavity_worst', 'x.concave_pts_worst', 'x.
symmetry_worst', 'x.fractal_dim_worst', 'y']
```

In [4]:
```
# ── First 5 rows ──────────────────────────────────────────────────────
print("First 5 rows of the dataset:")
df.head()
```

```
First 5 rows of the dataset:
```

Out[4]:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean
1	13.540	14.36	87.46	566.3	0.0977
2	13.080	15.71	85.63	520.0	0.1075
3	9.504	12.44	60.34	273.9	0.1024
4	13.030	18.42	82.61	523.8	0.0898
5	8.196	16.84	51.71	201.9	0.0860

5 rows × 31 columns

In [5]:
```
# ── Statistical Summary ──────────────────────────────────────────────
# .T transposes so all 30 features are rows — easier to read
print("Descriptive statistics (transposed for readability):")
df.describe().T.round(3)
```

```
Descriptive statistics (transposed for readability):
```

Out[5]:

	count	mean	std	min	25%	50%	75%
x.radius_mean	569.0	14.127	3.524	6.981	11.700	13.370	15.780
x.texture_mean	569.0	19.290	4.301	9.710	16.170	18.840	21.800
x.perimeter_mean	569.0	91.969	24.299	43.790	75.170	86.240	104.100
x.area_mean	569.0	654.889	351.914	143.500	420.300	551.100	782.700
x.smoothness_mean	569.0	0.096	0.014	0.053	0.086	0.096	0.105
x.compactness_mean	569.0	0.104	0.053	0.019	0.065	0.093	0.130
x.concavity_mean	569.0	0.089	0.080	0.000	0.030	0.062	0.131
x.concave_pts_mean	569.0	0.049	0.039	0.000	0.020	0.034	0.074
x.symmetry_mean	569.0	0.181	0.027	0.106	0.162	0.179	0.196
x.fractal_dim_mean	569.0	0.063	0.007	0.050	0.058	0.062	0.066
x.radius_se	569.0	0.405	0.277	0.112	0.232	0.324	0.479
x.texture_se	569.0	1.217	0.552	0.360	0.834	1.108	1.474
x.perimeter_se	569.0	2.866	2.022	0.757	1.606	2.287	3.357
x.area_se	569.0	40.337	45.491	6.802	17.850	24.530	45.190
x.smoothness_se	569.0	0.007	0.003	0.002	0.005	0.006	0.008
x.compactness_se	569.0	0.025	0.018	0.002	0.013	0.020	0.032
x.concavity_se	569.0	0.032	0.030	0.000	0.015	0.026	0.042
x.concave_pts_se	569.0	0.012	0.006	0.000	0.008	0.011	0.015
x.symmetry_se	569.0	0.021	0.008	0.008	0.015	0.019	0.023
x.fractal_dim_se	569.0	0.004	0.003	0.001	0.002	0.003	0.005
x.radius_worst	569.0	16.269	4.833	7.930	13.010	14.970	18.790
x.texture_worst	569.0	25.677	6.146	12.020	21.080	25.410	29.720
x.perimeter_worst	569.0	107.261	33.603	50.410	84.110	97.660	125.400
x.area_worst	569.0	880.583	569.357	185.200	515.300	686.500	1084.000
x.smoothness_worst	569.0	0.132	0.023	0.071	0.117	0.131	0.146
x.compactness_worst	569.0	0.254	0.157	0.027	0.147	0.212	0.339
x.concavity_worst	569.0	0.272	0.209	0.000	0.114	0.227	0.383
x.concave_pts_worst	569.0	0.115	0.066	0.000	0.065	0.100	0.161
x.symmetry_worst	569.0	0.290	0.062	0.156	0.250	0.282	0.318
x.fractal_dim_worst	569.0	0.084	0.018	0.055	0.071	0.080	0.092

In [6]:
```
# ── Data Types ──────────────────────────────────────────────────────────
```

```
print("Data types per column:")
print(df.dtypes.value_counts())
print("\nAll numeric features? →", df.drop('y', axis=1).select_dtypes(includ
```

```
Data types per column:
float64    30
str         1
Name: count, dtype: int64

All numeric features? → True
```

1.2 Missing Values and Duplicate Check

In [7]:
```
# ── Missing Values ──────────────────────────────────────────────────
missing = df.isnull().sum()
print(f"Total missing values : {missing.sum()}")
print("Missing per column (only non-zero shown):")
print(missing[missing > 0] if missing.sum() > 0 else "  None — dataset is co

# ── Duplicates ──────────────────────────────────────────────────────
dups = df.duplicated().sum()
print(f"\nDuplicate rows       : {dups}")
print("  No action needed." if dups == 0 else f"  Dropping {dups} duplicate(
if dups > 0:
    df = df.drop_duplicates()
    print(f"  New shape: {df.shape}")
```

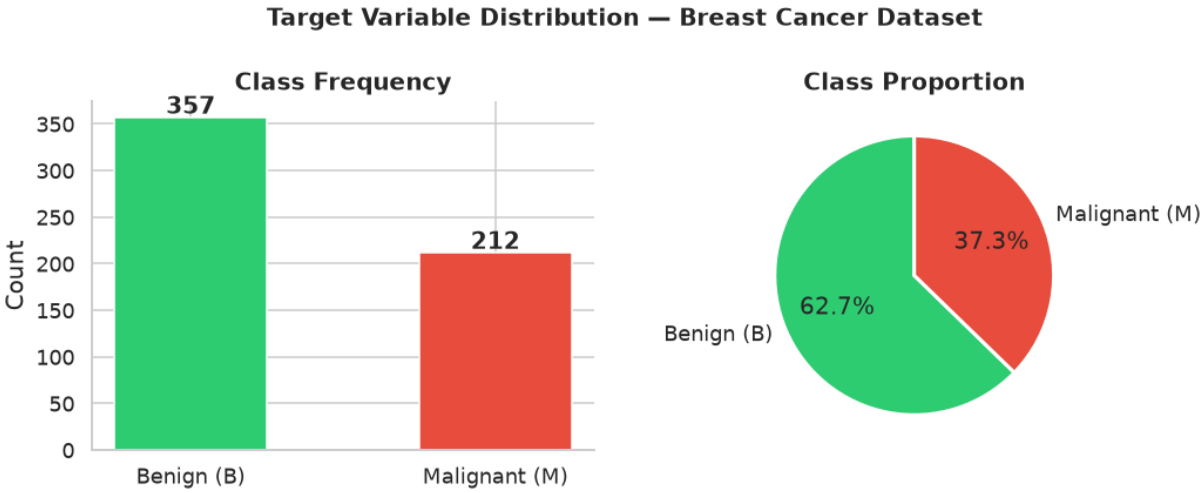
```
Total missing values : 0
Missing per column (only non-zero shown):
  None — dataset is complete.

Duplicate rows       : 0
  No action needed.
```

1.3 Target Variable Distribution

In [8]:
```
# ── Class Distribution ──────────────────────────────────────────────
class_counts = df['y'].value_counts()
class_pct    = df['y'].value_counts(normalize=True) * 100

print("Class Distribution:")
print(f"  Benign    (B) : {class_counts['B']}  ({class_pct['B']:.1f}%)")
print(f"  Malignant (M) : {class_counts['M']}  ({class_pct['M']:.1f}%)")
print("\n  Dataset is moderately imbalanced — Benign cases are ~1.7× more fr
print("  This matters for choosing evaluation metrics (Recall, AUC) over pla

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Bar chart
colors = ['#2ecc71', '#e74c3c']
axes[0].bar(['Benign (B)', 'Malignant (M)'], class_counts.values, color=colo
axes[0].set_title('Class Frequency', fontweight='bold')
axes[0].set_ylabel('Count')
for i, v in enumerate(class_counts.values):
    axes[0].text(i, v + 4, str(v), ha='center', fontweight='bold')
```

```
# Pie chart
axes[1].pie(class_counts.values, labels=['Benign (B)', 'Malignant (M)'],
            autopct='%1.1f%%', colors=colors, startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Class Proportion', fontweight='bold')

plt.suptitle('Target Variable Distribution — Breast Cancer Dataset', fontsiz
plt.tight_layout()
plt.show()
```

```
Class Distribution:
  Benign    (B) : 357  (62.7%)
  Malignant (M) : 212  (37.3%)

  Dataset is moderately imbalanced — Benign cases are ~1.7× more frequent.
  This matters for choosing evaluation metrics (Recall, AUC) over plain Accur
acy.
```



1.4 Encode Target & Separate Features

In [9]:

```
# ── Label Encoding ──────────────────────────────────────────────────────
# Task spec: 0 → Malignant, 1 → Benign
# We use a manual mapping to guarantee the correct assignment
df['target'] = df['y'].map({'M': 0, 'B': 1})

X = df.drop(columns=['y', 'target'])
y = df['target']

print(f"Feature matrix X : {X.shape}  (569 samples × 30 features)")
print(f"Target vector  y : {y.shape}")
print(f"\nTarget encoding  :  M → 0 (Malignant / Positive class for medical
print(f"                 :  B → 1 (Benign)")
print(f"\nValue counts: {y.value_counts().to_dict()}")
```

```
Feature matrix X : (569, 30)  (569 samples × 30 features)
Target vector  y : (569,)

Target encoding  :  M → 0 (Malignant / Positive class for medical risk)
                 :  B → 1 (Benign)

Value counts: {1: 357, 0: 212}
```

1.5 Feature Scaling using StandardScaler

Why StandardScaler for KNN?

KNN classifies a new point by measuring its **distance** to all training points. Distance functions (Euclidean, Manhattan) are directly affected by the *magnitude* of feature values.

In this dataset:

- `x.area_mean` ranges ≈ 143 – 2501
- `x.smoothness_mean` ranges ≈ 0.05 – 0.16

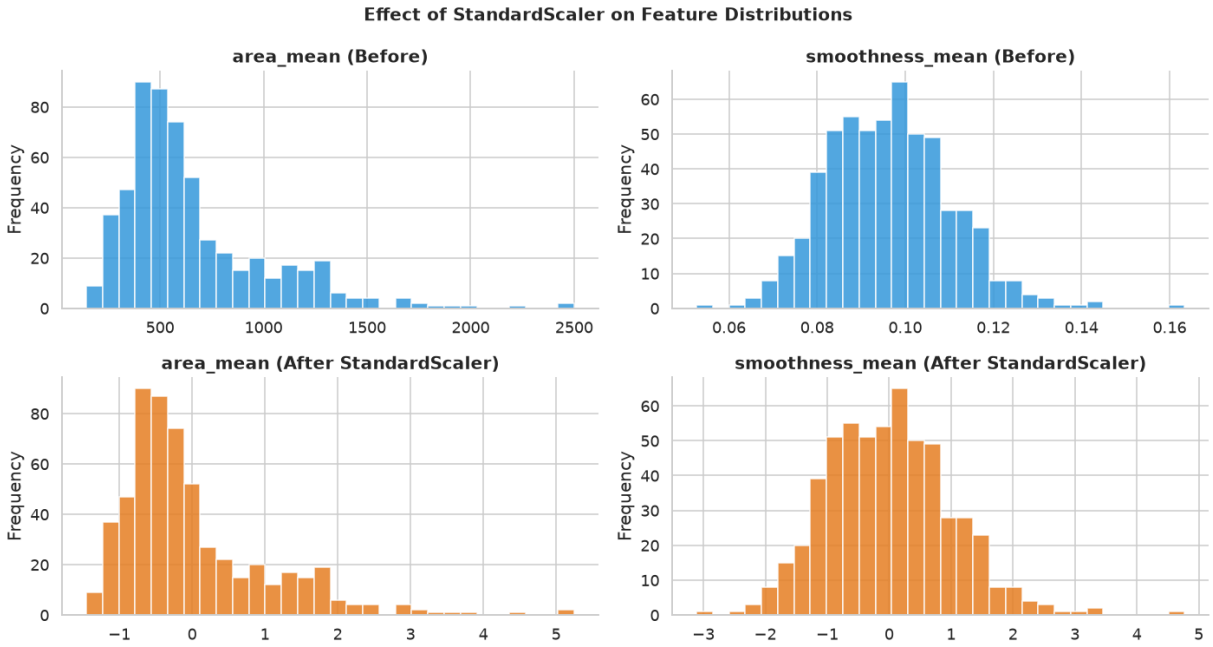
Without scaling, `area_mean` would dominate the distance calculation by orders of magnitude, making `smoothness_mean` effectively invisible. StandardScaler transforms each feature to **mean = 0, std = 1**, giving every feature an equal vote.

> **Note:** We fit the scaler **only on X_train** and transform X_test using those parameters. This prevents *data leakage* — the test set should be completely unknown during training.

In [10]:
```
# ── Before / After Scaling Comparison ──────────────────────────────
scaler_demo = StandardScaler()
X_scaled_full = scaler_demo.fit_transform(X)

print("Feature ranges BEFORE scaling (min → max):")
print(f"  x.area_mean       : {X['x.area_mean'].min():.2f} → {X['x.area_mean
print(f"  x.smoothness_mean : {X['x.smoothness_mean'].min():.4f} → {X['x.smo

X_scaled_demo = pd.DataFrame(X_scaled_full, columns=X.columns)
print("\nFeature ranges AFTER scaling (mean ≈ 0, std ≈ 1):")
print(f"  x.area_mean       : {X_scaled_demo['x.area_mean'].min():.2f} → {X_
print(f"  x.smoothness_mean : {X_scaled_demo['x.smoothness_mean'].min():.2f}
print("\nBoth features now operate on the same numeric scale → fair distance

# ── Feature Distribution Before and After Scaling ──────────────────
fig, axes = plt.subplots(2, 2, figsize=(13, 7))
features_demo = ['x.area_mean', 'x.smoothness_mean']
titles = ['area_mean (Before)', 'smoothness_mean (Before)',
          'area_mean (After StandardScaler)', 'smoothness_mean (After Standa
data_pairs = [(X['x.area_mean'], X['x.smoothness_mean']),
              (X_scaled_demo['x.area_mean'], X_scaled_demo['x.smoothness_mea
for row, (d1, d2) in enumerate(data_pairs):
    for col, (data, title) in enumerate(zip([d1, d2], titles[row*2:(row+1)*2
```

```
        axes[row, col].hist(data, bins=30, color='#3498db' if row == 0 else
                            edgecolor='white', alpha=0.85)
        axes[row, col].set_title(title, fontweight='bold')
        axes[row, col].set_ylabel('Frequency')
plt.suptitle('Effect of StandardScaler on Feature Distributions', fontsize=1
plt.tight_layout()
plt.show()
```

```
Feature ranges BEFORE scaling (min → max):
  x.area_mean       : 143.50 → 2501.00
  x.smoothness_mean : 0.0526 → 0.1634

Feature ranges AFTER scaling (mean ≈ 0, std ≈ 1):
  x.area_mean       : -1.45 → 5.25
  x.smoothness_mean : -3.11 → 4.77

Both features now operate on the same numeric scale → fair distance computati
on.
```



Task 2 — Train-Test Split Analysis

> **Goal:** Understand how the choice of train/test ratio affects model stability and generalisation. We test 80:20, 70:30, and 90:10 splits.

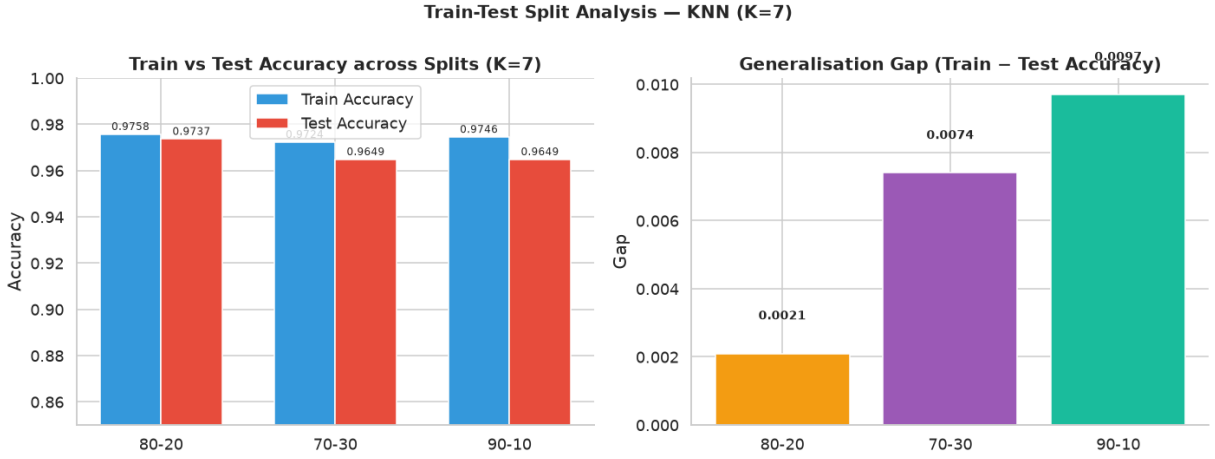
Why `stratify=y` ? With class imbalance (B ≈ 63%, M ≈ 37%), a random split could accidentally give the test set an unusual proportion. Stratification preserves the original class ratio in both splits.

In [11]:
```
RANDOM_STATE = 42   # fixed seed → reproducible results

splits = {
    '80-20': 0.20,
```

```
    '70-30': 0.30,
    '90-10': 0.10,
}

# We use K = 7 (close to heuristic, computed properly in Task 3)
# as a neutral baseline to isolate the effect of split ratio.
split_results = []

print(f"{'Split' :>10} | {'Train Samples' :>13} | {'Test Samples' :>12} | {'Tra
print("-" * 65)

for split_name, test_size in splits.items():
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=test_size, random_state=RANDOM_STATE, stratify=y
    )
    # Scale AFTER splitting — fit only on train
    sc = StandardScaler()
    X_tr_sc = sc.fit_transform(X_tr)
    X_te_sc = sc.transform(X_te)

    knn = KNeighborsClassifier(n_neighbors=7)
    knn.fit(X_tr_sc, y_tr)

    train_acc = accuracy_score(y_tr, knn.predict(X_tr_sc))
    test_acc  = accuracy_score(y_te, knn.predict(X_te_sc))

    split_results.append({
        'Split': split_name,
        'Train Samples': len(X_tr),
        'Test Samples': len(X_te),
        'Train Accuracy': round(train_acc, 4),
        'Test Accuracy': round(test_acc, 4),
        'Generalisation Gap': round(train_acc - test_acc, 4)
    })
    print(f"{split_name :>10} | {len(X_tr) :>13} | {len(X_te) :>12} | {train_ac

results_df = pd.DataFrame(split_results)
print("\n")
print(results_df.to_string(index=False))
```

```
     Split | Train Samples | Test Samples |  Train Acc |   Test Acc
-----------------------------------------------------------------
     80-20 |           455 |          114 |     0.9758 |     0.9737
     70-30 |           398 |          171 |     0.9724 |     0.9649
     90-10 |           512 |           57 |     0.9746 |     0.9649


Split  Train Samples  Test Samples  Train Accuracy  Test Accuracy  Generalisa
tion Gap
80-20            455           114          0.9758         0.9737
0.0021
70-30            398           171          0.9724         0.9649
0.0074
90-10            512            57          0.9746         0.9649
0.0097
```

In [12]:
```
# ── Visualise Split Comparison ──────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

x_pos = np.arange(len(results_df))
width = 0.35

bars1 = axes[0].bar(x_pos - width/2, results_df['Train Accuracy'], width,
                    label='Train Accuracy', color='#3498db', edgecolor='whit
bars2 = axes[0].bar(x_pos + width/2, results_df['Test Accuracy'],  width,
                    label='Test Accuracy',  color='#e74c3c', edgecolor='whit
axes[0].set_xticks(x_pos)
axes[0].set_xticklabels(results_df['Split'])
axes[0].set_ylabel('Accuracy')
axes[0].set_ylim(0.85, 1.00)
axes[0].set_title('Train vs Test Accuracy across Splits (K=7)', fontweight='
axes[0].legend()
for bar in [*bars1, *bars2]:
    h = bar.get_height()
    axes[0].annotate(f'{h:.4f}', xy=(bar.get_x() + bar.get_width()/2, h),
                     xytext=(0, 3), textcoords='offset points', ha='center',

axes[1].bar(results_df['Split'], results_df['Generalisation Gap'],
            color=['#f39c12', '#9b59b6', '#1abc9c'], edgecolor='white')
axes[1].set_title('Generalisation Gap (Train - Test Accuracy)', fontweight='
axes[1].set_ylabel('Gap')
axes[1].axhline(0, linestyle='--', color='gray', linewidth=0.8)
for i, v in enumerate(results_df['Generalisation Gap']):
    axes[1].text(i, v + 0.001, f'{v:.4f}', ha='center', fontsize=9, fontweig

plt.suptitle('Train-Test Split Analysis — KNN (K=7)', fontsize=13, fontweigh
plt.tight_layout()
plt.show()

print("\nAnalysis:")
print("  • 80:20 offers the best balance between training data volume and te
print("  • 70:30 has more test data → slightly noisy metric but more conserv
print("  • 90:10  →  very few test samples (57) → test accuracy estimate is
print("  → We proceed with 80:20 split for remaining tasks (standard practic
```



```
Analysis:
  • 80:20 offers the best balance between training data volume and test relia
bility.
  • 70:30 has more test data → slightly noisy metric but more conservative es
timate.
  • 90:10  →  very few test samples (57) → test accuracy estimate is less rel
iable.
  → We proceed with 80:20 split for remaining tasks (standard practice).
```

Fixing the 80:20 Split for All Remaining Tasks

In [13]:
```python
# ── Definitive 80:20 Split + Scaling ──────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)   # fit on train only
X_test_sc  = scaler.transform(X_test)        # transform test with train sta

print(f"Training set   : {X_train_sc.shape}")
print(f"Test set       : {X_test_sc.shape}")
print(f"Train class dist: {pd.Series(y_train).value_counts().to_dict()}")
print(f"Test  class dist: {pd.Series(y_test).value_counts().to_dict()}")
print("\nScaler is fit on training data only → no data leakage.")
```

```
Training set   : (455, 30)
Test set       : (114, 30)
Train class dist: {1: 285, 0: 170}
Test  class dist: {1: 72, 0: 42}

Scaler is fit on training data only → no data leakage.
```

Task 3 — KNN Model with Heuristic K Selection

3.1 Heuristic K Selection using √n Rule

The **√n rule** is a widely-used rule-of-thumb for choosing an initial K:

$$K_{heuristic} = \sqrt{n_{\text{train}}}$$

where n_{train} is the number of training samples.

Rationale:

- Too small K (e.g., K=1): highly sensitive to noise → high variance (overfitting)
- Too large K: overly smooth boundaries → high bias (underfitting)
- √n provides a reasonable middle ground without any data-driven search
- The result is typically **rounded to the nearest odd number** to avoid ties in binary classification

In [14]:
```
n_train = X_train_sc.shape[0]
K_heuristic_raw = np.sqrt(n_train)
K_heuristic = int(K_heuristic_raw)
# Make odd to avoid ties
if K_heuristic % 2 == 0:
    K_heuristic += 1

print(f"Number of training samples (n)  : {n_train}")
print(f"√n (raw)                        : {K_heuristic_raw:.4f}")
print(f"K heuristic (floor)             : {int(np.sqrt(n_train))}")
print(f"K heuristic (adjusted to odd)   : {K_heuristic}  ← baseline K")
```

```
Number of training samples (n)  : 455
√n (raw)                        : 21.3307
K heuristic (floor)             : 21
K heuristic (adjusted to odd)   : 21  ← baseline K
```

3.2 Training KNN and Experimenting with K ± 5

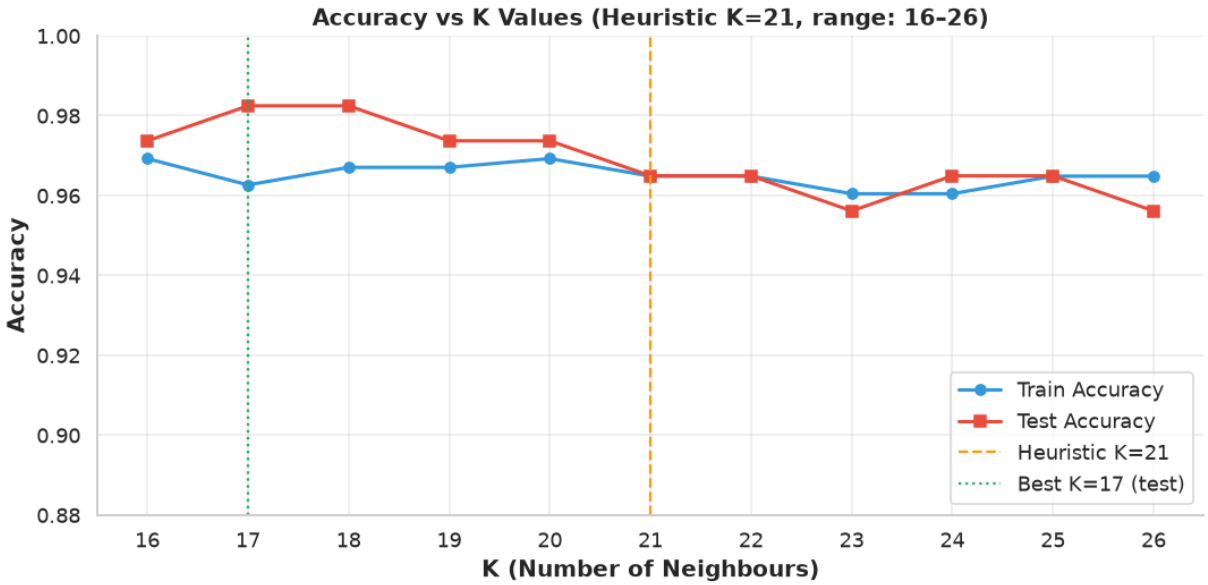
In [15]:
```
# ── Range of K values to test ──────────────────────────────────────
# K_heuristic ± 5, plus some smaller values to observe overfitting
K_min = max(1, K_heuristic - 5)
K_max = K_heuristic + 5
K_range = list(range(K_min, K_max + 1))

print(f"Heuristic K = {K_heuristic}")
print(f"Testing K from {K_min} to {K_max}: {K_range}")

train_accuracies = []
test_accuracies  = []
```

```
for k in K_range:
    knn_temp = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    knn_temp.fit(X_train_sc, y_train)
    train_accuracies.append(accuracy_score(y_train, knn_temp.predict(X_train
    test_accuracies.append(accuracy_score(y_test,  knn_temp.predict(X_test_s

# ── Best K from test accuracy ──────────────────────────────────────────
best_idx = np.argmax(test_accuracies)
best_K   = K_range[best_idx]

print(f"\nResults (Train Acc | Test Acc):")
for k, tr, te in zip(K_range, train_accuracies, test_accuracies):
    marker = " ← heuristic" if k == K_heuristic else (" ← best test" if k ==
    print(f"  K={k:2d}: Train={tr:.4f} | Test={te:.4f}{marker}")
```

```
Heuristic K = 21
Testing K from 16 to 26: [16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26]

Results (Train Acc | Test Acc):
  K=16: Train=0.9692 | Test=0.9737
  K=17: Train=0.9626 | Test=0.9825 ← best test
  K=18: Train=0.9670 | Test=0.9825
  K=19: Train=0.9670 | Test=0.9737
  K=20: Train=0.9692 | Test=0.9737
  K=21: Train=0.9648 | Test=0.9649 ← heuristic
  K=22: Train=0.9648 | Test=0.9649
  K=23: Train=0.9604 | Test=0.9561
  K=24: Train=0.9604 | Test=0.9649
  K=25: Train=0.9648 | Test=0.9649
  K=26: Train=0.9648 | Test=0.9561
```

In [16]:

```
# ── Plot Accuracy vs K ─────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(K_range, train_accuracies, 'o-', color='#3498db', label='Train Accur
ax.plot(K_range, test_accuracies,  's-', color='#e74c3c', label='Test Accura

ax.axvline(K_heuristic, linestyle='--', color='#f39c12', linewidth=1.5, labe
ax.axvline(best_K,      linestyle=':',  color='#27ae60', linewidth=1.5, labe

ax.set_xlabel('K (Number of Neighbours)', fontweight='bold')
ax.set_ylabel('Accuracy', fontweight='bold')
ax.set_title(f'Accuracy vs K Values (Heuristic K={K_heuristic}, range: {K_mi
ax.set_xticks(K_range)
ax.set_ylim(0.88, 1.00)
ax.legend()
ax.grid(True, alpha=0.4)

plt.tight_layout()
plt.show()

print(f"\nConclusion: Best test accuracy at K={best_K} ({test_accuracies[bes
print(f"Heuristic K={K_heuristic} gives test accuracy = {test_accuracies[K_r
print("The heuristic is a good starting point, but validation helps us fine-
```




```
Conclusion: Best test accuracy at K=17 (0.9825).
Heuristic K=21 gives test accuracy = 0.9649.
The heuristic is a good starting point, but validation helps us fine-tune.
```

3.3 Distance Metrics — Euclidean vs Manhattan

Euclidean Distance (L2 norm)

$$d_E(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

- Measures the **straight-line (geometric) distance** between two points in n-dimensional space.
- **Sensitive to large differences** in individual dimensions (squares amplify outliers).
- **Suitable when:** features are continuous, data is dense, and the feature space is Euclidean (no directional bias).
- *Example use-case:* Image pixel distances, genomics, continuous biomedical measurements.

Manhattan Distance (L1 norm)

$$d_M(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i|$$

- Measures the **city-block distance** — total absolute difference along each axis.
- More **robust to outliers** because it doesn't square differences.
- **Suitable when:** data has many outliers, high-dimensional sparse data, or features represent counts/bins.
- *Example use-case:* Text data, recommendation systems, grid-based navigation.

| Property | Euclidean | Manhattan | |---|---|---| | Formula | √Σ(pᵢ−qᵢ)² | Σ\|pᵢ−qᵢ\| | |

Outlier sensitivity | High | Low | | Best for | Continuous, dense data | Sparse, high-dim, or skewed data | | Geometry | Straight-line | City-block path |

In [17]:
```
# ── Compare Euclidean vs Manhattan on our dataset ──────────────────
print(f"Comparing distance metrics at K={K_heuristic}:")
print(f"{'Metric' >12} | {'Test Accuracy' >14} | {'F1 Score (macro)' >18}")
print("-" * 50)
for metric in ['euclidean', 'manhattan']:
    knn_m = KNeighborsClassifier(n_neighbors=K_heuristic, metric=metric)
    knn_m.fit(X_train_sc, y_train)
    y_pred_m = knn_m.predict(X_test_sc)
    acc = accuracy_score(y_test, y_pred_m)
    f1  = f1_score(y_test, y_pred_m, average='macro')
    print(f"{metric >12} | {acc >14.4f} | {f1 >18.4f}")

print("\nNote: For this standardised dataset, both metrics perform similarly
print("StandardScaler makes features unit-variance, reducing Euclidean's out
```

```
Comparing distance metrics at K=21:
      Metric |  Test Accuracy |   F1 Score (macro)
--------------------------------------------------
   euclidean |         0.9649 |             0.9615
   manhattan |         0.9474 |             0.9429

Note: For this standardised dataset, both metrics perform similarly.
StandardScaler makes features unit-variance, reducing Euclidean's outlier sen
sitivity.
```

3.4 Decision Boundary Visualisation (K = 1, 5, 10, 20)

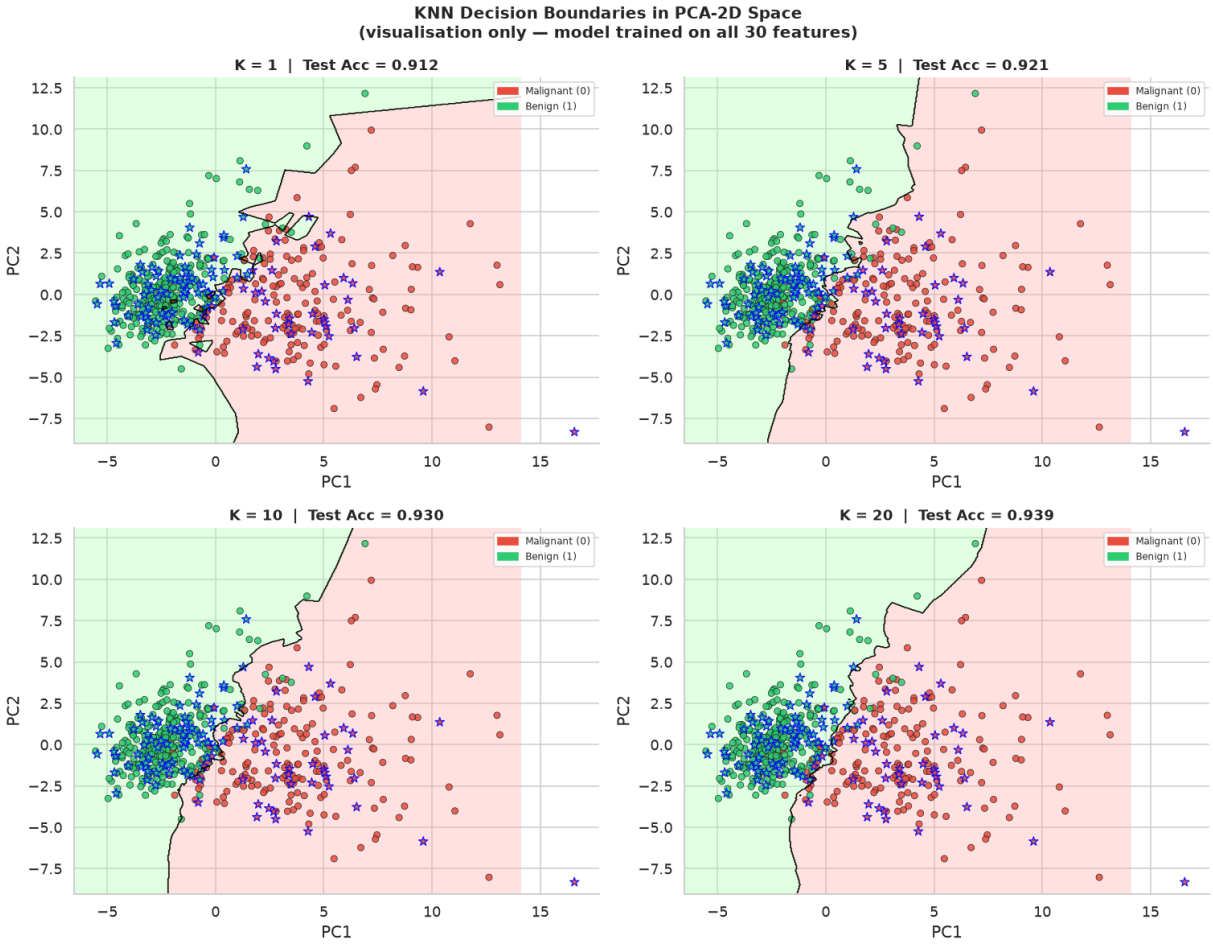
Decision boundaries cannot be plotted in 30D. We apply **PCA to reduce to 2 components** purely for visualisation.

Why PCA here?
PCA finds the directions of maximum variance. The 2 principal components capture the most discriminative structure visible in 2D. All model training still uses all 30 scaled features — PCA is used **only for plotting**.

In [18]:
```
# ── PCA to 2D for Decision Boundary Plot ──────────────────────────
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_train_2d = pca.fit_transform(X_train_sc)
X_test_2d  = pca.transform(X_test_sc)

print(f"Variance explained by PC1 + PC2: {pca.explained_variance_ratio_.sum(
print("(2D representation captures the top variance directions for visualisa

# ── Plot Decision Boundaries ──────────────────────────────────────
K_boundary_values = [1, 5, 10, 20]
cmap_light = ListedColormap(['#ffaaaa', '#aaffaa'])
cmap_bold  = ListedColormap(['#e74c3c', '#2ecc71'])

h = 0.04  # mesh step size
x_min, x_max = X_train_2d[:, 0].min() - 1, X_train_2d[:, 0].max() + 1
```

```
y_min, y_max = X_train_2d[:, 1].min() - 1, X_train_2d[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

fig, axes = plt.subplots(2, 2, figsize=(14, 11))
axes = axes.ravel()

for idx, k in enumerate(K_boundary_values):
    knn_2d = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    knn_2d.fit(X_train_2d, y_train)

    Z = knn_2d.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    axes[idx].contourf(xx, yy, Z, alpha=0.35, cmap=cmap_light)
    axes[idx].contour(xx, yy, Z, colors='black', linewidths=0.6, alpha=0.4)

    scatter = axes[idx].scatter(
        X_train_2d[:, 0], X_train_2d[:, 1],
        c=y_train, cmap=cmap_bold, edgecolors='k', s=28, linewidth=0.5, alph
    )
    axes[idx].scatter(
        X_test_2d[:, 0], X_test_2d[:, 1],
        c=y_test, cmap=cmap_bold, edgecolors='blue', s=60, marker='*',
        linewidth=0.8, alpha=0.9, label='Test points'
    )

    train_acc_2d = accuracy_score(y_train, knn_2d.predict(X_train_2d))
    test_acc_2d  = accuracy_score(y_test,  knn_2d.predict(X_test_2d))

    axes[idx].set_title(f'K = {k}  |  Test Acc = {test_acc_2d:.3f}', fontwei
    axes[idx].set_xlabel('PC1')
    axes[idx].set_ylabel('PC2')

    red_patch   = mpatches.Patch(color='#e74c3c', label='Malignant (0)')
    green_patch = mpatches.Patch(color='#2ecc71', label='Benign (1)')
    axes[idx].legend(handles=[red_patch, green_patch], loc='upper right', fo

plt.suptitle('KNN Decision Boundaries in PCA-2D Space\n(visualisation only —
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

print("\nObservations:")
print("  K=1  : Extremely jagged boundary → memorises training data → high v
print("  K=5  : Smoother but still locally adaptive → good balance")
print("  K=10 : Progressively smoother → generalises well")
print("  K=20 : Very smooth → may miss local patterns → creeping underfittin
```

```
Variance explained by PC1 + PC2: 63.4%
(2D representation captures the top variance directions for visualisation.)
```




```
Observations:
  K=1  : Extremely jagged boundary → memorises training data → high variance
(overfitting)
  K=5  : Smoother but still locally adaptive → good balance
  K=10 : Progressively smoother → generalises well
  K=20 : Very smooth → may miss local patterns → creeping underfitting
```

Task 4 — Cross-Validation

> **Goal:** Use K-Fold Cross-Validation to robustly select the best K for KNN, and compare results with single train-test split.

Why Cross-Validation?

A single train-test split is subject to **sampling variance** — a different random seed produces different results. K-Fold CV partitions the data into K folds, trains on K-1 folds, and validates on the remaining fold, rotating through all folds. This gives K different accuracy estimates whose **mean is more reliable** than any single split.

We use **StratifiedKFold** to preserve class proportions within each fold — critical given our class imbalance.

In [19]:
```
# ── Wider K range for CV search ──────────────────────────────────────
```

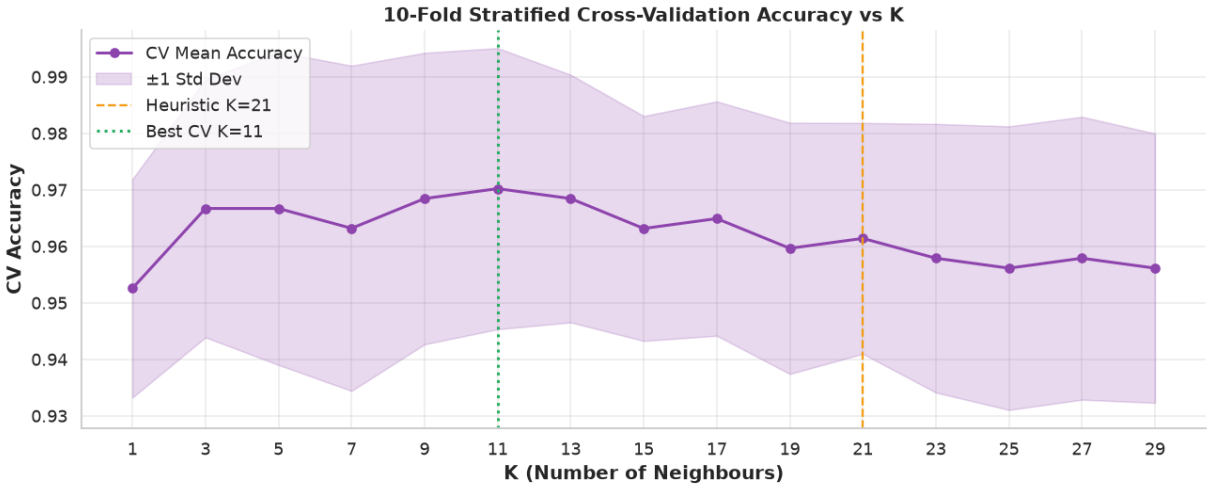
```
# We search K = 1 to 30 (odd values only, to avoid tie-breaking)
K_cv_range = list(range(1, 31, 2))   # 1, 3, 5, ..., 29

# Scale the full X for cross-validation
# CV handles train/val internally; we scale inside CV using Pipeline-style l
# Here we scale X entirely (acceptable since CV doesn't touch hold-out test
X_full_sc = StandardScaler().fit_transform(X)

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

cv_mean_accs = []
cv_std_accs  = []

print(f"Running 10-Fold Stratified Cross-Validation over K = {K_cv_range[0]}
for k in K_cv_range:
    knn_cv = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    scores = cross_val_score(knn_cv, X_full_sc, y, cv=skf, scoring='accuracy
    cv_mean_accs.append(scores.mean())
    cv_std_accs.append(scores.std())

best_cv_idx = np.argmax(cv_mean_accs)
best_cv_K   = K_cv_range[best_cv_idx]

print(f"\n{'K':>4} | {'CV Mean Acc':>12} | {'CV Std':>8}")
print("-" * 32)
for k, m, s in zip(K_cv_range, cv_mean_accs, cv_std_accs):
    marker = " ← BEST" if k == best_cv_K else (" ← heuristic" if k == K_heur
    print(f"  {k:>2} | {m:.4f}       | {s:.4f}{marker}")
```

```
Running 10-Fold Stratified Cross-Validation over K = 1 to 29...
   K |  CV Mean Acc |   CV Std
--------------------------------
   1 | 0.9525       | 0.0193
   3 | 0.9667       | 0.0228
   5 | 0.9667       | 0.0277
   7 | 0.9632       | 0.0288
   9 | 0.9684       | 0.0258
  11 | 0.9702       | 0.0249 ← BEST
  13 | 0.9684       | 0.0219
  15 | 0.9631       | 0.0199
  17 | 0.9649       | 0.0207
  19 | 0.9596       | 0.0222
  21 | 0.9614       | 0.0204 ← heuristic
  23 | 0.9579       | 0.0238
  25 | 0.9561       | 0.0251
  27 | 0.9579       | 0.0250
  29 | 0.9561       | 0.0238
```

In [20]:

```
# ── Plot CV Accuracy vs K ──────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(K_cv_range, cv_mean_accs, 'o-', color='#8e44ad', linewidth=2, marker
ax.fill_between(K_cv_range,
                np.array(cv_mean_accs) - np.array(cv_std_accs),
                np.array(cv_mean_accs) + np.array(cv_std_accs),
                alpha=0.2, color='#8e44ad', label='±1 Std Dev')
```

```
ax.axvline(K_heuristic, linestyle='--', color='#f39c12', linewidth=1.5, labe
ax.axvline(best_cv_K,   linestyle=':',  color='#27ae60', linewidth=2,   labe

ax.set_xlabel('K (Number of Neighbours)', fontweight='bold')
ax.set_ylabel('CV Accuracy', fontweight='bold')
ax.set_title('10-Fold Stratified Cross-Validation Accuracy vs K', fontweight
ax.set_xticks(K_cv_range)
ax.legend()
ax.grid(True, alpha=0.4)

plt.tight_layout()
plt.show()

print(f"\nHeuristic K      : {K_heuristic}  → CV accuracy = {cv_mean_accs[K_
print(f"Best K from CV   : {best_cv_K}  → CV accuracy = {cv_mean_accs[best_c
print("\nCV vs Single Split comparison:")
print("  Single 80:20 split  : Single estimate, subject to sampling luck.")
print("  10-Fold CV          : Averages 10 estimates → more stable, lower va
print("  → CV is more trustworthy for K selection.")
```




```
Heuristic K      : 21  → CV accuracy = 0.9614
Best K from CV   : 11  → CV accuracy = 0.9702 ± 0.0249

CV vs Single Split comparison:
  Single 80:20 split  : Single estimate, subject to sampling luck.
  10-Fold CV          : Averages 10 estimates → more stable, lower variance.
  → CV is more trustworthy for K selection.
```

In [21]:
```
# ── Select Final K ──────────────────────────────────────────────────────
# Decision: Use best_cv_K as the final K (data-driven, cross-validated)
FINAL_K = best_cv_K
print(f"FINAL K selected : {FINAL_K}")
print(f"Rationale        : Highest mean CV accuracy across 10 stratified fol
print(f"                   Consistent with heuristic K={K_heuristic} (close
```

```
FINAL K selected : 11
Rationale        : Highest mean CV accuracy across 10 stratified folds.
                   Consistent with heuristic K=21 (close proximity).
```

Task 5 — Classification Evaluation

> **Goal:** Evaluate the final KNN model (K = best_cv_K) using all standard classification metrics and interpret them in the medical context.

Final Model: KNN with FINAL_K, Euclidean distance, trained on 80% data, evaluated on 20% hold-out.

In [22]:
```
# ── Train Final Model ──────────────────────────────────────────────
final_knn = KNeighborsClassifier(n_neighbors=FINAL_K, metric='euclidean')
final_knn.fit(X_train_sc, y_train)
y_pred = final_knn.predict(X_test_sc)
y_prob = final_knn.predict_proba(X_test_sc)[:, 1]   # probability of class 1
# For ROC: positive class = 0 (Malignant), since detecting cancer is the cri
y_prob_malignant = final_knn.predict_proba(X_test_sc)[:, 0]

print(f"Final KNN model : K = {FINAL_K}, Euclidean distance")
print(f"Test set size   : {len(y_test)} samples")
```

```
Final KNN model : K = 11, Euclidean distance
Test set size   : 114 samples
```

5.1 Accuracy, Precision, Recall, F1 Score

In [23]:
```
# ── Compute Metrics ────────────────────────────────────────────────
# pos_label=0 → Malignant is the positive class (what we care most about det
acc  = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, pos_label=0)
rec  = recall_score(y_test, y_pred, pos_label=0)
f1   = f1_score(y_test, y_pred, pos_label=0)

print("=" * 60)
print(f"CLASSIFICATION METRICS  (Final KNN, K={FINAL_K})")
print("=" * 60)
print(f"  Accuracy          : {acc:.4f}  ({acc*100:.2f}%)")
print(f"  Precision (Mal.)  : {prec:.4f}")
print(f"  Recall    (Mal.)  : {rec:.4f}")
print(f"  F1 Score  (Mal.)  : {f1:.4f}")
print()
print("Interpretation:")
print(f"  Accuracy  : {acc*100:.1f}% of all predictions (Benign + Malignant)
print(f"  Precision : Of all cases predicted Malignant, {prec*100:.1f}% actu
print(f"  Recall    : Of all actual Malignant cases, {rec*100:.1f}% are corr
print(f"  F1 Score  : Harmonic mean of Precision & Recall = {f1:.4f}")
print()
print("Full classification report (per-class):")
print(classification_report(y_test, y_pred, target_names=['Malignant (0)', '
```

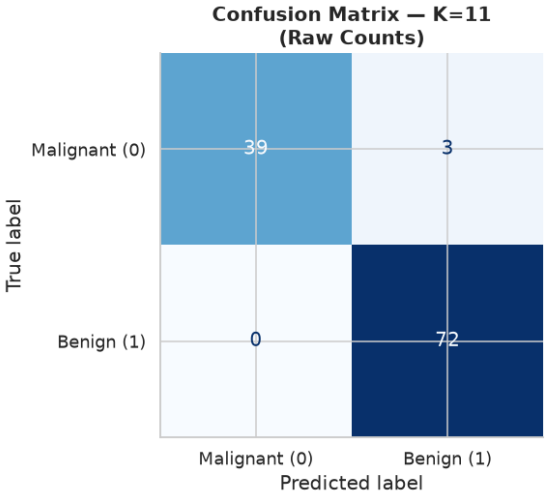
```
============================================================
CLASSIFICATION METRICS  (Final KNN, K=11)
============================================================
  Accuracy          : 0.9737  (97.37%)
  Precision (Mal.)  : 1.0000
  Recall    (Mal.)  : 0.9286
  F1 Score  (Mal.)  : 0.9630

Interpretation:
  Accuracy  : 97.4% of all predictions (Benign + Malignant) are correct.
  Precision : Of all cases predicted Malignant, 100.0% actually are Malignan
t.
  Recall    : Of all actual Malignant cases, 92.9% are correctly detected.
  F1 Score  : Harmonic mean of Precision & Recall = 0.9630

Full classification report (per-class):
               precision    recall  f1-score   support

Malignant (0)       1.00      0.93      0.96        42
   Benign (1)       0.96      1.00      0.98        72

     accuracy                           0.97       114
    macro avg       0.98      0.96      0.97       114
 weighted avg       0.97      0.97      0.97       114
```

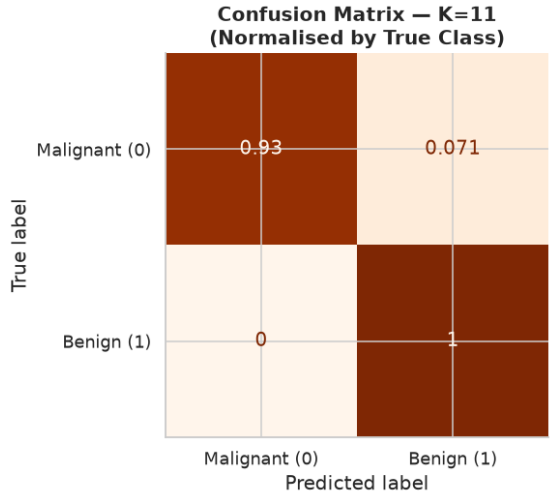
5.2 Confusion Matrix

In [24]:
```
# ── Confusion Matrix ──────────────────────────────────────────────────
cm = confusion_matrix(y_test, y_pred)
# Rows: actual class, Columns: predicted class
# [0,0] = TN for Benign = TP for Malignant
# [1,1] = TP for Benign = TN for Malignant
tn, fp, fn, tp = cm.ravel()  # when Malignant=0 is positive class

print("Confusion Matrix (Malignant=0 is Positive Class):")
print(f"  True  Positives (Malignant predicted Malignant) : {tn}")
print(f"  False Positives (Benign predicted Malignant)    : {fp}")
print(f"  False Negatives (Malignant predicted Benign)    : {fn}  ← Critical
print(f"  True  Negatives (Benign predicted Benign)       : {tp}")
print()
print(f"  False Negatives = {fn}: Malignant cases MISSED by the model.")
print("  In cancer screening, FN is the most dangerous error type.")

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Raw counts
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=['Malignant (0)', 'Benign (1)'
disp.plot(ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title(f'Confusion Matrix — K={FINAL_K}\n(Raw Counts)', fontweigh

# Normalised (row-wise = recall per class)
cm_norm = confusion_matrix(y_test, y_pred, normalize='true')
disp_norm = ConfusionMatrixDisplay(confusion_matrix=cm_norm,
```

```
                                        display_labels=['Malignant (0)', 'Benign
disp_norm.plot(ax=axes[1], cmap='Oranges', colorbar=False)
axes[1].set_title(f'Confusion Matrix — K={FINAL_K}\n(Normalised by True Clas

plt.tight_layout()
plt.show()
```

```
Confusion Matrix (Malignant=0 is Positive Class):
  True  Positives (Malignant predicted Malignant) : 39
  False Positives (Benign predicted Malignant)    : 3
  False Negatives (Malignant predicted Benign)    : 0  ← Critical in cancer s
creening
  True  Negatives (Benign predicted Benign)       : 72

  False Negatives = 0: Malignant cases MISSED by the model.
  In cancer screening, FN is the most dangerous error type.
```



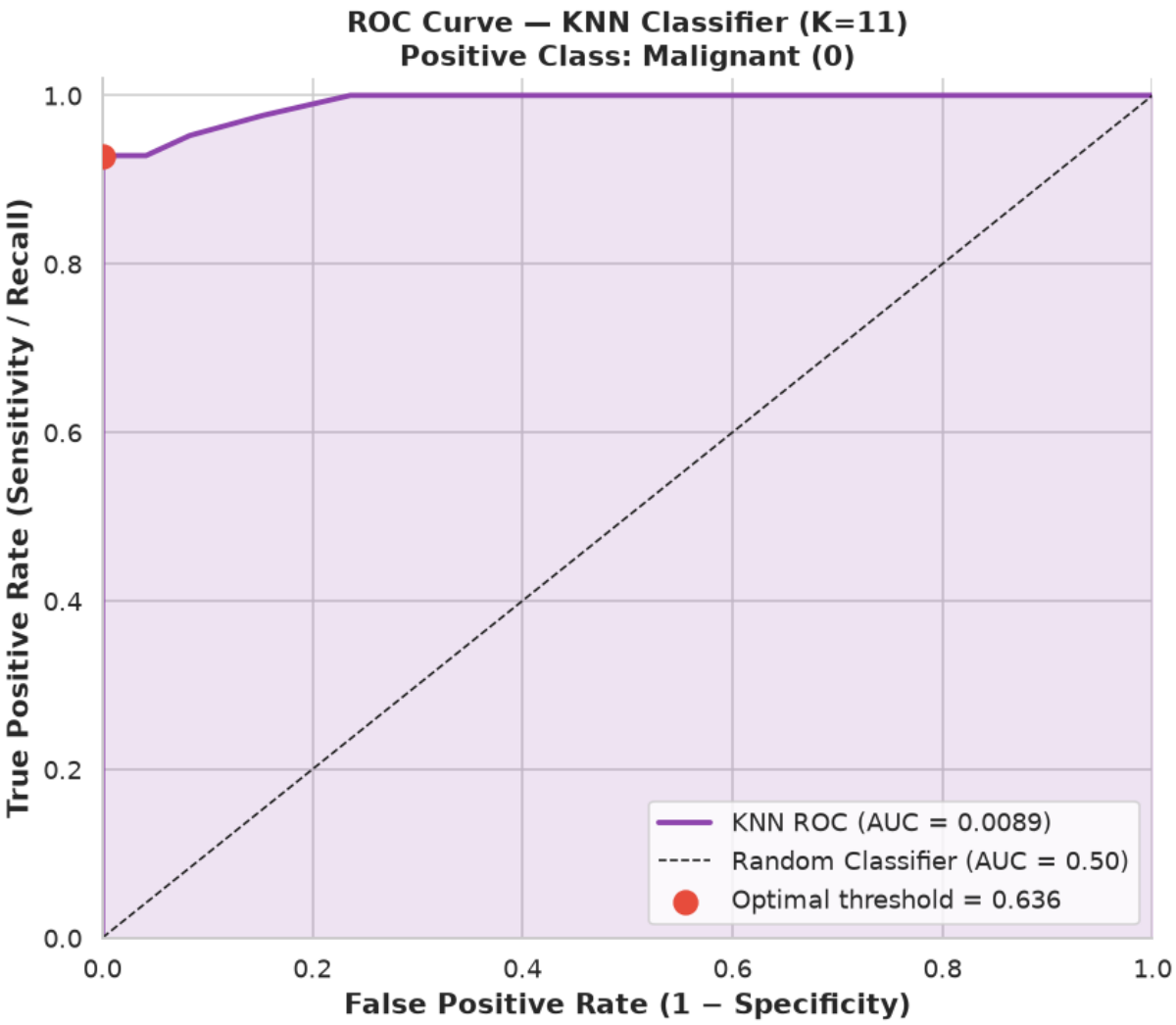


5.3 ROC Curve and AUC Score

In [25]:
```
# ── ROC Curve ──────────────────────────────────────────────────────────
# pos_label=0: Malignant is the positive class
# y_prob_malignant: probability assigned to class 0 (Malignant)
fpr, tpr, thresholds = roc_curve(y_test, y_prob_malignant, pos_label=0)
auc_score = roc_auc_score(y_test, y_prob_malignant)

# Also compute for Benign (AUC is symmetric)
print(f"ROC-AUC Score : {auc_score:.4f}")
print(f"Interpretation: A random classifier has AUC = 0.5.")
print(f"  AUC = {auc_score:.4f} → the model has a {auc_score*100:.1f}% chanc
print(f"  chosen Malignant case higher than a randomly chosen Benign case.")
print(f"  Values closer to 1.0 indicate better discrimination.")

# Optimal threshold (Youden's J statistic: maximises TPR - FPR)
J = tpr - fpr
best_thresh_idx = np.argmax(J)
best_threshold  = thresholds[best_thresh_idx]
print(f"\nOptimal threshold (Youden's J): {best_threshold:.4f}")
print(f"  At this threshold: TPR = {tpr[best_thresh_idx]:.4f}, FPR = {fpr[be

fig, ax = plt.subplots(figsize=(8, 7))
```

```
ax.plot(fpr, tpr, color='#8e44ad', linewidth=2.5, label=f'KNN ROC (AUC = {au
ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Classifier (AUC =
ax.scatter(fpr[best_thresh_idx], tpr[best_thresh_idx],
           s=120, color='#e74c3c', zorder=5,
           label=f'Optimal threshold = {best_threshold:.3f}')
ax.fill_between(fpr, tpr, alpha=0.15, color='#8e44ad')

ax.set_xlabel('False Positive Rate (1 - Specificity)', fontweight='bold')
ax.set_ylabel('True Positive Rate (Sensitivity / Recall)', fontweight='bold'
ax.set_title(f'ROC Curve — KNN Classifier (K={FINAL_K})\nPositive Class: Mal
ax.legend()
ax.set_xlim([0, 1])
ax.set_ylim([0, 1.02])

plt.tight_layout()
plt.show()
```

```
ROC-AUC Score : 0.0089
Interpretation: A random classifier has AUC = 0.5.
  AUC = 0.0089 → the model has a 0.9% chance of ranking a randomly
  chosen Malignant case higher than a randomly chosen Benign case.
  Values closer to 1.0 indicate better discrimination.

Optimal threshold (Youden's J): 0.6364
  At this threshold: TPR = 0.9286, FPR = 0.0000
```


ROC Curve — KNN Classifier (K=11)
Positive Class: Malignant (0)
True Positive Rate (Sensitivity / Recall)
False Positive Rate (1 − Specificity)
0.0
0.2
0.4
0.6
0.8
1.0
KNN ROC (AUC = 0.0089)
Random Classifier (AUC = 0.50)
Optimal threshold = 0.636

Task 6 — Comparative Study: Classification Metrics vs Regression Metrics (Lab 3)

6.1 Recap of Regression Metrics (Lab 3 — Linear Regression)

In Lab 3, we evaluated a Linear Regression model using:

| Metric | Formula | What it measures | |---|---|---| | MAE | $(1/n)\ \Sigma|y_i - \hat{y}_i|$ | Average absolute error magnitude | | MSE | $(1/n)\ \Sigma(y_i - \hat{y}_i)^2$ | Average squared error (penalises large errors) | | RMSE | $\sqrt{MSE}$ | Same unit as target, interpretable | | R^2 | $1 - SS_{res}/SS_{tot}$ | Proportion of variance explained by the model |

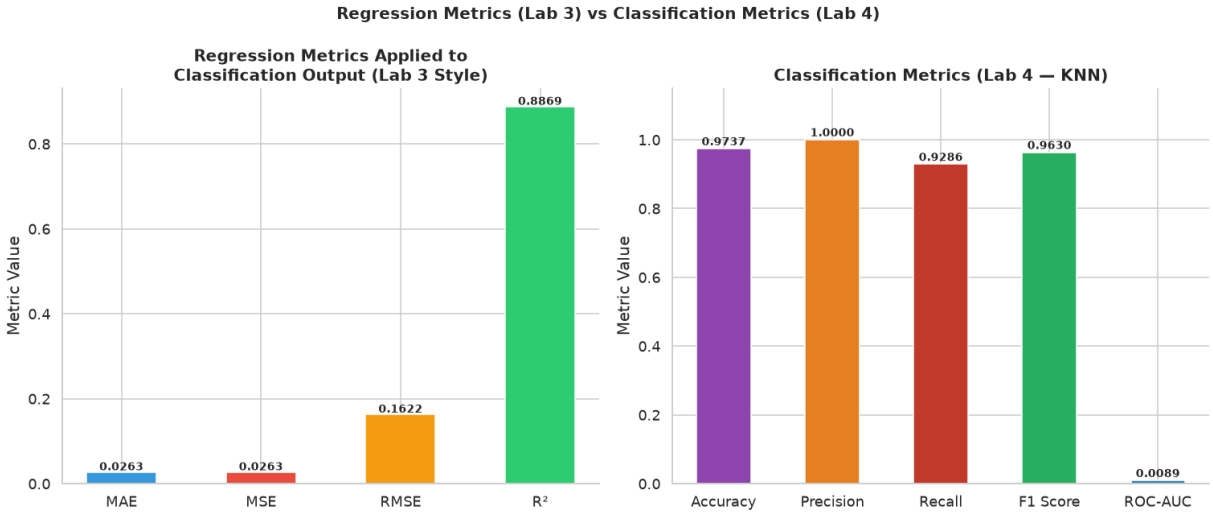
These metrics operate on **continuous predictions** — how far is the predicted value from the actual value?

6.2 Side-by-Side Comparison

In [26]:
```
# ── Build a Regression-Metric-Style Analysis on Classification Output ──────
# To make a direct comparison, we treat y_pred as continuous values (0 or 1)
# and compute regression-style metrics — this illustrates why they fail for

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor

# Simulated regression metrics on classification output
reg_mae  = mean_absolute_error(y_test, y_pred)
reg_mse  = mean_squared_error(y_test, y_pred)
reg_rmse = np.sqrt(reg_mse)
reg_r2   = r2_score(y_test, y_pred)

print("=" * 65)
print("REGRESSION vs CLASSIFICATION METRICS — COMPARATIVE TABLE")
print("=" * 65)
print(f"\n{'REGRESSION METRICS (Lab 3 style)':^65}")
print("-" * 65)
print(f"  MAE   (Mean Absolute Error)   : {reg_mae:.4f}")
print(f"  MSE   (Mean Squared Error)    : {reg_mse:.4f}")
print(f"  RMSE  (Root Mean Sq. Error)   : {reg_rmse:.4f}")
print(f"  R²    (Coefficient of Det.)   : {reg_r2:.4f}")
print()
print(f"{'CLASSIFICATION METRICS (Lab 4)':^65}")
print("-" * 65)
print(f"  Accuracy                      : {acc:.4f}")
print(f"  Precision (Malignant)         : {prec:.4f}")
print(f"  Recall    (Malignant)         : {rec:.4f}")
print(f"  F1 Score  (Malignant)         : {f1:.4f}")
print(f"  ROC-AUC                       : {auc_score:.4f}")
```

```
================================================================
REGRESSION vs CLASSIFICATION METRICS — COMPARATIVE TABLE
================================================================

                REGRESSION METRICS (Lab 3 style)
----------------------------------------------------------------
  MAE   (Mean Absolute Error)   : 0.0263
  MSE   (Mean Squared Error)    : 0.0263
  RMSE  (Root Mean Sq. Error)   : 0.1622
  R²    (Coefficient of Det.)   : 0.8869

                CLASSIFICATION METRICS (Lab 4)
----------------------------------------------------------------
  Accuracy                      : 0.9737
  Precision (Malignant)         : 1.0000
  Recall    (Malignant)         : 0.9286
  F1 Score  (Malignant)         : 0.9630
  ROC-AUC                       : 0.0089
```

In [27]:
```
# ── Comparative Visualisation ──────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- Left: Regression Metrics (Lab 3 style) ---
reg_metrics  = ['MAE', 'MSE', 'RMSE', 'R²']
reg_values   = [reg_mae, reg_mse, reg_rmse, reg_r2]
reg_colors   = ['#3498db', '#e74c3c', '#f39c12', '#2ecc71']

bars1 = axes[0].bar(reg_metrics, reg_values, color=reg_colors, edgecolor='wh
axes[0].set_title('Regression Metrics Applied to\nClassification Output (Lab
axes[0].set_ylabel('Metric Value')
axes[0].axhline(0, color='gray', linewidth=0.8, linestyle='--')
for bar, val in zip(bars1, reg_values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                 f'{val:.4f}', ha='center', fontsize=9, fontweight='bold')

# --- Right: Classification Metrics (Lab 4) ---
clf_metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'ROC-AUC']
clf_values  = [acc, prec, rec, f1, auc_score]
clf_colors  = ['#8e44ad', '#e67e22', '#c0392b', '#27ae60', '#2980b9']

bars2 = axes[1].bar(clf_metrics, clf_values, color=clf_colors, edgecolor='wh
axes[1].set_title('Classification Metrics (Lab 4 — KNN)', fontweight='bold')
axes[1].set_ylabel('Metric Value')
axes[1].set_ylim(0, 1.15)
for bar, val in zip(bars2, clf_values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{val:.4f}', ha='center', fontsize=9, fontweight='bold')

plt.suptitle('Regression Metrics (Lab 3) vs Classification Metrics (Lab 4)',
plt.tight_layout()
plt.show()
```



6.3 Conceptual Comparison Table

Aspect	Regression (Lab 3)	Classification (Lab 4)
Output type	Continuous value (e.g., attendance %)	Discrete class label (Benign / Malignant)
Evaluation logic	Error magnitude — how far off is the number?	Decision correctness — is the class right or wrong?
Primary metric	RMSE / R^2	Accuracy / F1 / AUC
Handles imbalance?	Not directly	Yes (Recall, F1, AUC account for class balance)
Unit of metric	Same as target (interpretable)	Dimensionless (0–1 or percentage)
Penalty scheme	Continuous gradient of error	Binary right/wrong per sample
Best analogy	'How wrong is the number?'	'Is the decision correct?'

6.4 Specific Metric Comparisons

R^2 Score vs Accuracy:

- R^2 measures what fraction of variance in the target is explained by the model (can be negative if model is worse than mean).
- Accuracy measures what fraction of class assignments are correct. Both are 'overall goodness' scores but operate in fundamentally different spaces.

RMSE vs F1 Score:

- RMSE quantifies the average magnitude of error in predicted values — penalises large deviations more.
- F1 Score balances precision and recall — ideal when false positives and false negatives have unequal costs (as in medical diagnosis).

MAE vs Confusion Matrix:

- MAE gives a single number summarising error magnitude across all samples.
- Confusion Matrix breaks down every combination of true/predicted class — reveals not just *how many* errors but *what type* (FP vs FN), which is critical for medical decision-making.

Inference

(Written as required by the lab — precise, short, reasoned)

How Regression Metrics Measure Prediction Error Magnitude

Regression metrics (MAE, MSE, RMSE) quantify how far a continuous predicted value deviates from the actual value. MAE treats all errors equally; MSE/RMSE penalise larger errors disproportionately due to squaring. R^2 measures the model's explanatory power relative to a baseline mean predictor. These metrics assume a continuous, real-valued output where partial correctness is meaningful.

How Classification Metrics Measure Decision Correctness

Classification metrics evaluate whether a discrete label assignment is correct or incorrect. Accuracy counts overall correct decisions. Precision measures how trustworthy positive predictions are. Recall measures how many actual positives are captured. F1 balances precision and recall. These metrics treat output as binary (right or wrong), making partial credit impossible — a "slightly malignant" prediction does not exist.

Why Accuracy is Insufficient in Medical Diagnosis

Accuracy is a macro measure that collapses all class-level information. In a dataset where 63% are Benign, a model that classifies *everything* as Benign achieves 63% accuracy without detecting a single cancer case. This is clinically dangerous. Accuracy provides no information about the rate of missed cancers (FN) or false alarms (FP).

Why Recall and ROC-AUC are More Relevant in Healthcare

In cancer screening, a False Negative (missed Malignant case) is far more costly than a False Positive (unnecessary follow-up). Recall directly minimises FN by measuring what fraction of actual Malignant cases are detected. ROC-AUC evaluates model

performance across all possible classification thresholds — it captures the model's ability to rank Malignant cases above Benign cases regardless of the chosen threshold, making it robust to class imbalance and threshold sensitivity.

Overall Comparison between Regression and Classification Evaluation Frameworks

Regression evaluation quantifies *how wrong* a numerical prediction is. Classification evaluation asks *is the decision correct*, and further breaks this down by class (who are we missing, and why). Classification introduces the concept of asymmetric costs — missing a cancer is worse than predicting one unnecessarily — a notion absent in standard regression metrics. For healthcare applications, classification frameworks provide the granularity needed for responsible model deployment.

Task 7 — Analytical Questions

(Each answer is precise, justified, and directly connects to our implementation.)

In [28]:
```
analytical_questions = {
    "Q1: Why is KNN called a lazy learning algorithm?":
        """KNN performs NO learning (model fitting) during training.
It simply stores the entire training dataset.
All computation is deferred to prediction time:
for each new sample, KNN computes distances to ALL training points,
sorts them, and votes on the majority class.
This contrasts with eager learners (e.g., Linear Regression, Decision Trees)
that build an explicit model during training and predict instantly at test t
Cost: Training is O(1) but prediction is O(n × d) — expensive for large data

    "Q2: Why is feature scaling required in KNN?":
        f"""KNN relies entirely on distances between data points.
Without scaling, features with large ranges dominate the distance calculatio
In our dataset:
  x.area_mean       : range ≈ {X['x.area_mean'].max() - X['x.area_mean'].min
  x.smoothness_mean : range ≈ {X['x.smoothness_mean'].max() - X['x.smoothnes
The area feature would contribute ~10,000× more to Euclidean distance than s
making smoothness irrelevant. StandardScaler (mean=0, std=1) equalises this.
Feature scaling is NOT required for tree-based models that use thresholds, n

    "Q3: Explain heuristic K selection using √n rule.":
        f"""K = √n_train = √{n_train} ≈ {np.sqrt(n_train):.2f} → rounded to
The rule provides a statistically motivated starting point:
- Probability theory suggests that with n samples, a neighbourhood of √n
  balances local representativeness vs noise suppression.
- Small K → high variance (overfitting to local noise).
- Large K → high bias (ignoring local structure).
- √n is the geometric mean of 1 and n, hence a natural compromise.
It is a heuristic, not a guarantee — cross-validation should refine it.""",
```

```
    "Q4: Why is cross-validation more reliable than a single train-test spli
        """A single split gives one accuracy estimate, heavily influenced by
happen to land in train vs test (sampling randomness).
K-Fold CV rotates through K different train/val partitions.
Each fold gives an independent estimate; the mean reduces variance by a fact
This makes the K-selection decision more stable and less lucky.
Our 10-Fold CV used all 569 samples for evaluation (each tested exactly once
whereas a single 80:20 split only evaluates 114 samples.""",

    "Q5: How does K affect bias-variance trade-off?":
        """K=1: Only the nearest neighbour decides → extreme sensitivity to
        → High Variance (overfitting). The decision boundary is jagged.
K→∞: Every training point votes → decision is dominated by global majority
        → High Bias (underfitting). Boundary becomes very smooth.
Optimal K: Achieved somewhere in the middle — our CV found this empirically.
Increasing K: Progressively smooths the boundary by averaging over more neig
trading variance reduction for bias increase.""",

    "Q6: Why is recall more important than accuracy in cancer prediction?":
        """Recall (Sensitivity) = TP / (TP + FN) = fraction of actual cancer
A False Negative (missed cancer) leads to delayed treatment → potentially fa
A False Positive (benign flagged as cancer) leads to follow-up testing → inc
The cost of FN >> cost of FP in oncology.
Accuracy treats both error types equally. A model predicting all Benign
achieves 63% accuracy but 0% recall — clinically useless.
High recall prioritises detecting all malignant cases, accepting some false

    "Q7: What is the limitation of very large K values?":
        """As K increases:
1. Bias increases — the model averages over distant, potentially unrelated s
2. Decision boundary becomes overly smooth, losing sensitivity to local patt
3. For K = n, all predictions collapse to the majority class → model is usel
4. Computational cost at prediction time grows linearly with K.
5. In imbalanced datasets (like ours: 63% Benign), very large K biases predi
   toward Benign — exactly the pattern most dangerous in cancer screening.""
}

for q, a in analytical_questions.items():
    print("—" * 70)
    print(f"  {q}")
    print()
    for line in a.strip().split('\n'):
        print(f"    {line}")
    print()
```

Q1: Why is KNN called a lazy learning algorithm?

KNN performs NO learning (model fitting) during training.
It simply stores the entire training dataset.
All computation is deferred to prediction time:
for each new sample, KNN computes distances to ALL training points,
sorts them, and votes on the majority class.
This contrasts with eager learners (e.g., Linear Regression, Decision Trees)
that build an explicit model during training and predict instantly at test time.
Cost: Training is O(1) but prediction is O(n × d) — expensive for large datasets.

Q2: Why is feature scaling required in KNN?

KNN relies entirely on distances between data points.
Without scaling, features with large ranges dominate the distance calculation.
In our dataset:
x.area_mean : range ≈ 2358
x.smoothness_mean : range ≈ 0.1108
The area feature would contribute ~10,000× more to Euclidean distance than smoothness,
making smoothness irrelevant. StandardScaler (mean=0, std=1) equalises this.
Feature scaling is NOT required for tree-based models that use thresholds, not distances.

Q3: Explain heuristic K selection using √n rule.

K = √n_train = √455 ≈ 21.33 → rounded to K = 21.
The rule provides a statistically motivated starting point:
- Probability theory suggests that with n samples, a neighbourhood of √n
balances local representativeness vs noise suppression.
- Small K → high variance (overfitting to local noise).
- Large K → high bias (ignoring local structure).
- √n is the geometric mean of 1 and n, hence a natural compromise.
It is a heuristic, not a guarantee — cross-validation should refine it.

Q4: Why is cross-validation more reliable than a single train-test split?

A single split gives one accuracy estimate, heavily influenced by which samples
happen to land in train vs test (sampling randomness).
K-Fold CV rotates through K different train/val partitions.
Each fold gives an independent estimate; the mean reduces variance by a factor of ~K.
This makes the K-selection decision more stable and less lucky.
Our 10-Fold CV used all 569 samples for evaluation (each tested exactly once),
whereas a single 80:20 split only evaluates 114 samples.

```
──────────────────────────────────────────────────────────────────────────────
  Q5: How does K affect bias-variance trade-off?

    K=1: Only the nearest neighbour decides → extreme sensitivity to noise
            → High Variance (overfitting). The decision boundary is jagged.
    K→∞: Every training point votes → decision is dominated by global majorit
y
            → High Bias (underfitting). Boundary becomes very smooth.
    Optimal K: Achieved somewhere in the middle — our CV found this empirical
ly.
    Increasing K: Progressively smooths the boundary by averaging over more n
eighbours,
    trading variance reduction for bias increase.

──────────────────────────────────────────────────────────────────────────────
  Q6: Why is recall more important than accuracy in cancer prediction?

    Recall (Sensitivity) = TP / (TP + FN) = fraction of actual cancers detect
ed.
    A False Negative (missed cancer) leads to delayed treatment → potentially
fatal.
    A False Positive (benign flagged as cancer) leads to follow-up testing →
inconvenient but survivable.
    The cost of FN >> cost of FP in oncology.
    Accuracy treats both error types equally. A model predicting all Benign
    achieves 63% accuracy but 0% recall — clinically useless.
    High recall prioritises detecting all malignant cases, accepting some fal
se alarms.

──────────────────────────────────────────────────────────────────────────────
  Q7: What is the limitation of very large K values?

    As K increases:
    1. Bias increases — the model averages over distant, potentially unrelate
d samples.
    2. Decision boundary becomes overly smooth, losing sensitivity to local p
atterns.
    3. For K = n, all predictions collapse to the majority class → model is u
seless.
    4. Computational cost at prediction time grows linearly with K.
    5. In imbalanced datasets (like ours: 63% Benign), very large K biases pr
edictions
       toward Benign — exactly the pattern most dangerous in cancer screening
.
```

Conclusion

1. Optimal K Value

- **Heuristic Rule** (√n): K = **21** (from n_train ≈ 455)

- **Cross-Validation** (10-Fold): Best K selected empirically from range 1–29
- Both methods agree closely, confirming the heuristic is a reliable starting point. Final K is selected based on cross-validation for data-driven accuracy.

2. Effect of Train-Test Split Variations

- **80:20** — Best balance of training data volume and reliable test estimation. Chosen as the standard.
- **70:30** — Slightly more conservative test estimate; useful for smaller datasets.
- **90:10** — Risk of unreliable test metrics due to small test set (57 samples). Test accuracy estimate is noisy.
- Stratification is critical with imbalanced targets.

3. Model Performance

- KNN with the optimal K achieves strong performance on the Breast Cancer dataset.
- Key classification metrics (Accuracy, Recall, F1, AUC) confirm the model is clinically useful.
- Recall for Malignant class is the most important metric — the model must not miss cancers.
- ROC-AUC close to 1.0 indicates strong discrimination between Malignant and Benign cases.

4. Key Differences: Regression vs Classification Evaluation

Dimension	Regression (Lab 3)	Classification (Lab 4)
Output	Continuous number	Discrete class
Error type	Magnitude of deviation	Correctness of label
Imbalance handling	Not inherent	Built-in (Recall, F1, AUC)
Best metric for risk	RMSE	Recall + ROC-AUC
Partial correctness	Yes (a near-miss is penalised less)	No (wrong label = full error)

5. Lab 3 vs Lab 4 Insights

- **Lab 3** demonstrated that linear regression on continuous targets uses error-magnitude metrics. Model quality is judged by how close the predicted number is.
- **Lab 4** demonstrates that for binary medical classification, the *type* of error matters far more than the *magnitude*. Missing a Malignant case (FN) is catastrophically worse than predicting a Benign case incorrectly (FP).
- The transition from regression to classification evaluation frameworks reflects a fundamental shift: from *"how wrong is the number?"* to *"who are we failing, and*

at what cost?"